30 Sep, 2025

```
struct name {      (book)

    int  a;
    char b;
} book1;
```

book1 → instance

properties اکٹھی کرے

multiple data types

enum → values اکٹھی کرے, only one data type, values assigned, if not written 0, or اس سے اگلی 3. اس datatype کے لحاظ سے بنتی جا ہے۔

Book

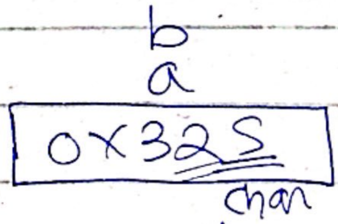
can be different sizes

→ seperate addresses

Union

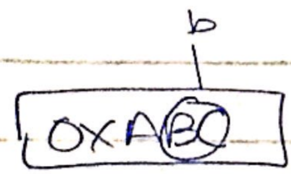
```
union book {
    int a;
    char b;
} book1;
```

سب سے بڑی datatype کا
size , ~~ca~~ refer ادھر کی
ہوتا ہے سب کو

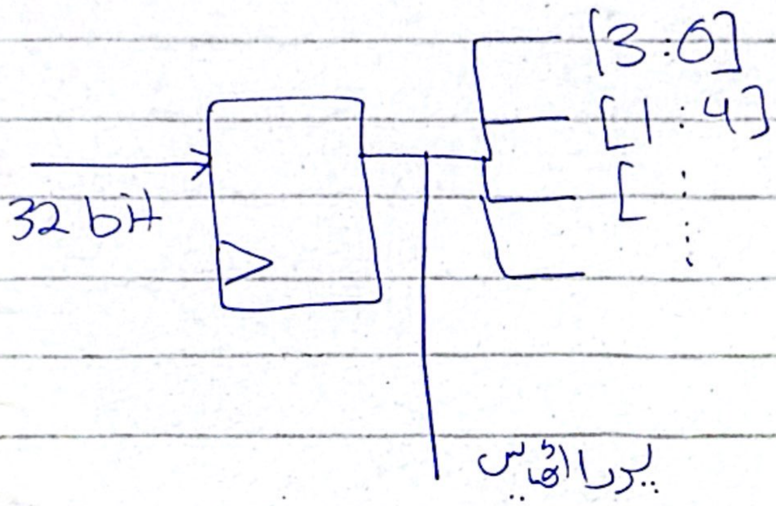


→ same addresses

book . a = 0XABC



ایک ہی جگہ پر پڑے تو ایک کو change کریں تو دوسرا change
ہو جائے گا



چھوٹے میں change کروں تو پورا والا / بڑا بھی change ہو جائے گا

اگر دو int بھی ہوتا تو بھی same رہتی -

Why Union over Struct?

→ Control and Status Reg of peripherals (some)

control → only write
status → only user can read

When writing then → control register
when reading → status register
by union ← give same memory

made such as in hardware

لکھتے وقت control کے طور پر access
پڑھتے وقت status کے طور پر access

Global

→ written at top, written in header files

Local

→ exists within a function scope

Static

```
function {
    static int a;   automatically initialized to 0
    a = a+2
}
```

→ ① will exist when reentered then 2 otherwise 1,1,1,1....

static local, static global

$\underbrace{\text{static local}}_{\text{function}}$, $\underbrace{\text{static global}}_{\text{global, file}}$

same { تو همین file
نه تو هر iteration بشه نمیشه -

<stdint.h>

uint_16t → short mapped
uint_32t → long mapped

→ code migration 32 → 16 (crash)
to avoid mad <stdint.h>

پس اگه طرح کدها رو از hardware جدا کنیم crash نمی‌کنه

my_header.h

```
#include <  >
#define
structs
enums
global variables
function declarations
```

write at top

```
#ifndef MY_HEADER
#define MY_HEADER_H


#endif //MY_HEADER_H
```

```
Char new_char,

new_char = 'A';   65

        new_char = 65;
```

ASCII table

```
                     STATE
typedef enum {

    IDLE = 00;  0
    DETECT_1 = 01;  1
    DETECT_2 = 10;  2
    DETECT_SEQ = 11;  3
}
```

```
typedef enum{
    IDLE;
    DETECT_1;
    DETECT_10;
    SEQ_DETECT;
  } state;

state   curr_state;

switch (curr_state)

    case( STATE IDLE):
        :
        break;

    case ( ~~STATE~~ DETECT 1):
        :
        break;
```

→ pointer to function ⇒ address of functions

High level language → no one to one mapping with binary

Low level → assembly (1 instruction 99% times maps to 32 bit (on architecture))

Linker → دوسروں سے تعلق بناتا